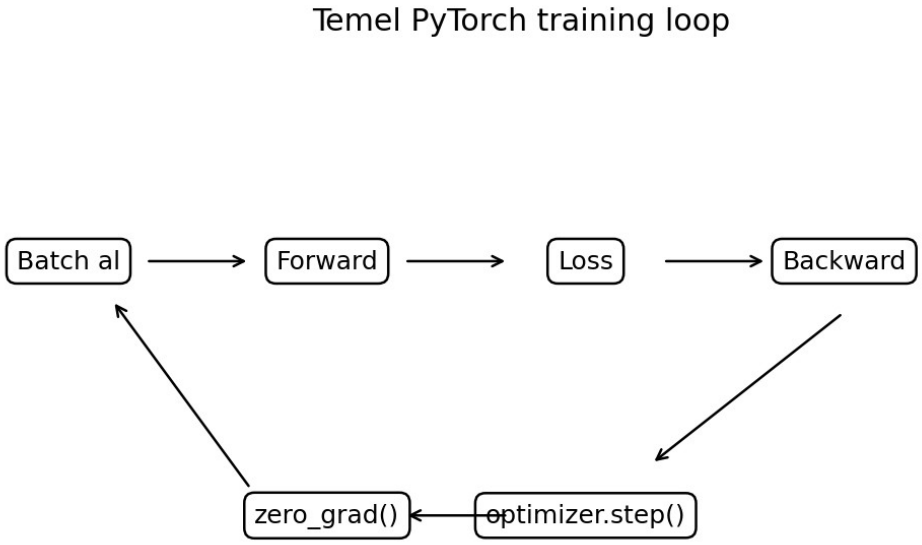
30. PYTORCH

Tensor • Dataset/DataLoader • nn.Module • Autograd • Training Loop • GPU



Bu modülün amacı
Deep Learning bilgisini PyTorch ile uygulamaya dönüştürmek: tensordan custom Dataset'e, model sınıfından tam training loop'a, GPU'dan checkpoint ve inference pipeline'ına kadar üretim mantığını öğrenmek.

Kazanım	Modül sonunda
Tensor	Shape, dtype, device ve temel tensor işlemlerini yönetebilirsin.
Data pipeline	Dataset ve DataLoader ile mini-batch veri akışı kurabilirsin.
Model	nn.Module ile reusable network yazabilirsin.
Training	Forward-loss-backward-step döngüsünü sıfırdan kurabilirsin.
GPU	Model ve batch'leri CUDA/MPS/CPU arasında taşıyabilirsin.
Production	Checkpoint, inference ve reproducible training düzeni kurabilirsin.

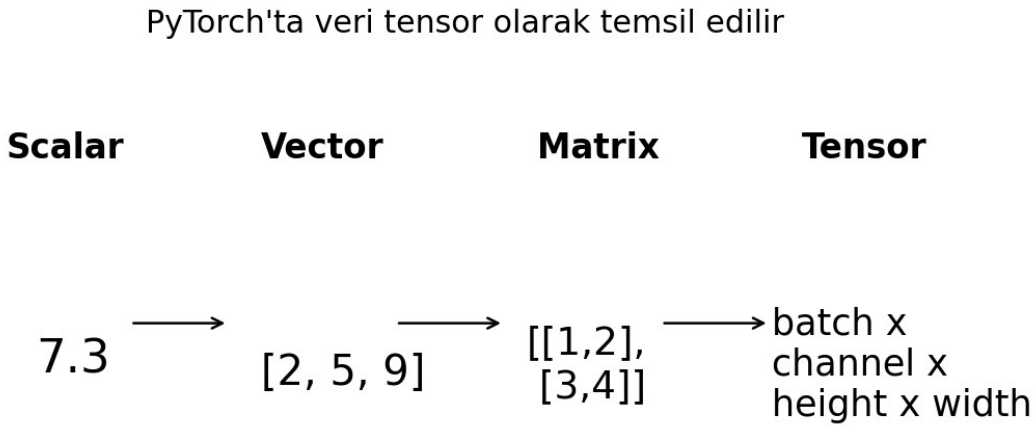
1. PyTorch nedir?

PyTorch, tensor computation, otomatik türev (autograd), neural network katmanları ve GPU hızlandırmayı tek API altında sağlayan bir deep learning framework'üdür.

Kavram	PyTorch karşılığı
Array	torch.Tensor
Neural layer	torch.nn
Automatic differentiation	torch.autograd
Optimizer	torch.optim
Mini-batch loader	DataLoader
Model class	nn.Module

Mental model
NumPy + autograd + GPU + neural-network building blocks şeklinde düşünebilirsin.

2. Tensor mantığı




```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
print(x.shape)
print(x.dtype)
print(x.device)
```

Her tensor için üç soru
Shape ne? Dtype ne? Hangi device üzerinde? PyTorch debugging'in büyük kısmı bu üç soruyla çözülür.

3. Tensor oluşturma

```
a = torch.zeros(3, 4)
b = torch.ones(2, 5)
c = torch.randn(32, 10)
d = torch.arange(0, 10)
e = torch.eye(4)
```

Fonksiyon	Amaç
zeros	0 dolu tensor
ones	1 dolu tensor
rand/randn	Random tensor
arange	Sequence
eye	Identity matrix
from_numpy	NumPy -> Tensor

4. Shape, reshape, view ve transpose

```
x = torch.randn(32, 3, 64, 64)

print(x.shape)

flat = x.reshape(32, -1)
transposed = x.permute(0, 2, 3, 1)
```

Batch dimension
Deep learning modellerinde ilk boyut çoğu zaman batch size'dır.

5. Broadcasting ve vectorization

PyTorch mümkün olduğunca Python loop yerine tensor operasyonları kullanır.

```
x = torch.randn(1000, 128)
bias = torch.randn(128)

y = x + bias
```

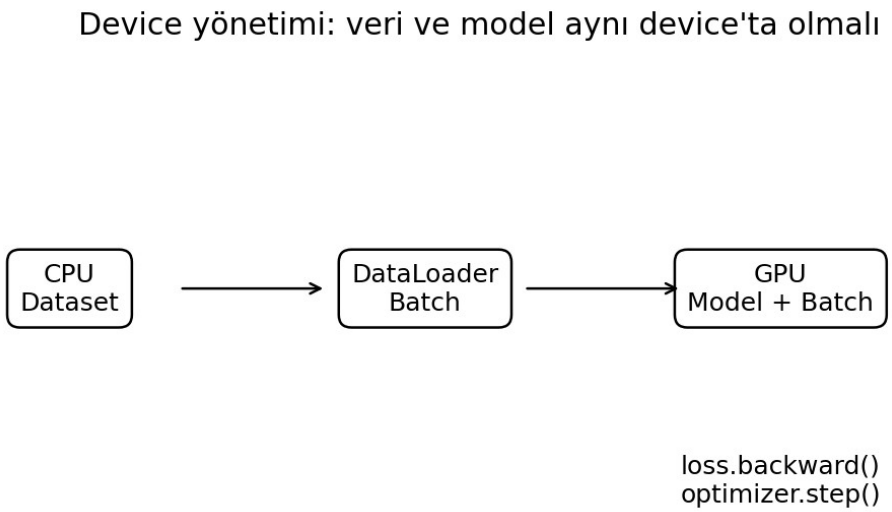
Performans
Vectorized tensor operasyonları GPU üzerinde büyük paralellik sağlar.

6. Dtype

Dtype	Tipik kullanım
float32	Default neural-network hesapları
float16/bfloat16	Mixed precision
float64	Yüksek hassasiyet gereken özel durumlar
int64 / long	Class index ve token id
bool	Mask

CrossEntropyLoss notu
Class label tensor'ı çoğunlukla torch.long dtype ister.

7. Device yönetimi




```
device = (
    "cuda" if torch.cuda.is_available()
    else "cpu"
)

x = x.to(device)
model = model.to(device)
```

Klasik hata
Model GPU'da ama input CPU'da ise PyTorch device mismatch hatası verir.

8. NumPy ile dönüşüm

```
import numpy as np

arr = np.array([1, 2, 3], dtype=np.float32)
x = torch.from_numpy(arr)

back = x.numpy()
```

Shared memory
CPU'da from_numpy ile oluşturulan tensor ve NumPy array bazı durumlarda aynı belleği paylaşabilir.

9. Autograd

Autograd, computation graph üzerinde gradient'leri otomatik hesaplar.

```
x = torch.tensor(
    3.0,
    requires_grad=True
)

y = x**2 + 2*x

y.backward()

print(x.grad)
```

Sonuç
dy/dx = 2x + 2 olduğundan x=3 için gradient 8 olur.

10. Computation graph

Forward sırasında tensor operasyonları bir graph oluşturur. backward() çağrısı loss'tan parametrelere doğru chain rule uygular.

- Leaf tensor
- requires_grad
- grad_fn
- backward()
- .grad

Backpropagation
PyTorch backpropagation matematiğini elle yazmaz; graph üzerinden otomatik türev alır.

11. no_grad ve inference_mode

```
model.eval()

with torch.inference_mode():
    preds = model(X_valid)
```

Neden?
Inference sırasında gradient graph oluşturmak gereksiz memory ve compute maliyeti yaratır.

12. nn.Module

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.net(x)
```

İki temel metot
__init__ içinde katmanları tanımla; forward içinde verinin akışını tanımla.

13. nn.Sequential

Katmanların düz ardışık olduğu modellerde kısa ve okunaklı yapı sağlar.

```
model = nn.Sequential(
    nn.Linear(20, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)
```

14. Parameters

```
for name, param in model.named_parameters():
    print(
        name,
        param.shape,
        param.requires_grad
    )
```

Trainable parametre
Optimizer yalnız requires_grad=True olan parameter'ları günceller.

15. Activation function'lar

Activation	Tipik kullanım
ReLU	Hidden layer default
GELU	Transformer ailesinde yaygın
Sigmoid	Binary probability / gating
Tanh	Bounded hidden representation
Softmax	Multiclass probability dağılımı

16. Loss function seçimi

Problem	Loss
Regression	MSELoss / L1Loss / SmoothL1Loss
Binary classification	BCEWithLogitsLoss
Multiclass classification	CrossEntropyLoss
Embedding similarity	Contrastive / cosine tabanlı loss

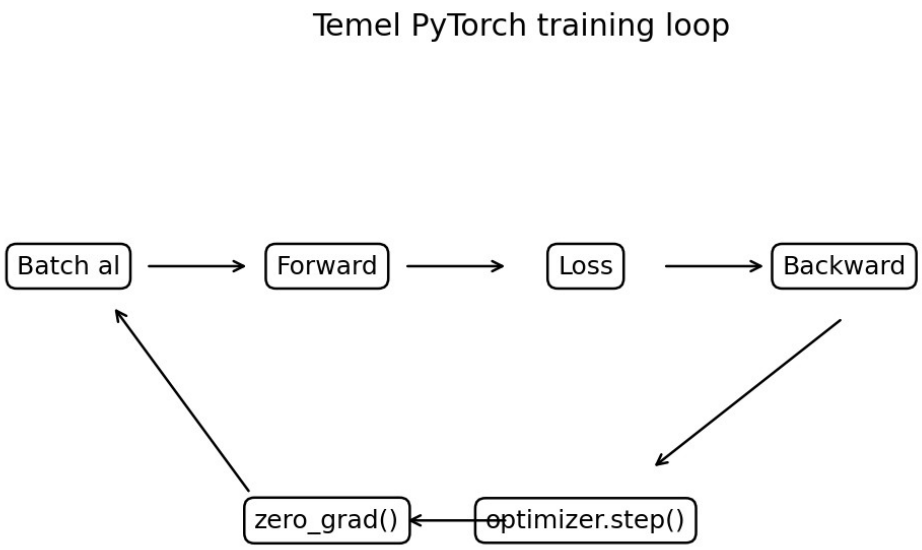
Logits
BCEWithLogitsLoss sigmoid'i, CrossEntropyLoss softmax mantığını loss içinde stabil biçimde uygular. Model sonunda ayrıca sigmoid/softmax koymak çoğu training senaryosunda gerekmez.

17. Optimizer

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    weight_decay=1e-4
)
```

Optimizer	Not
SGD	Basit ve güçlü; momentum ile
Adam	Adaptive learning rate
AdamW	Decoupled weight decay; modern default'lardan
RMSprop	Bazı sequence/vision senaryoları

18. Training loop




```
model.train()

for X_batch, y_batch in loader:
    X_batch = X_batch.to(device)
    y_batch = y_batch.to(device)

    optimizer.zero_grad()

    logits = model(X_batch)
    loss = criterion(logits, y_batch)

    loss.backward()
    optimizer.step()
```

Sıra önemlidir
zero_grad -> forward -> loss -> backward -> step, temel PyTorch training döngüsüdür.

19. Neden zero_grad?

PyTorch gradient'leri varsayılan olarak accumulate eder.

Eğer unutursan
Yeni batch gradient'i önceki batch gradient'inin üzerine eklenir. Bu bazen deliberate gradient accumulation için kullanılır, çoğu zaman ise bug'dır.

20. Dataset sınıfı

```
from torch.utils.data import Dataset

class TabularDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(
            X, dtype=torch.float32
        )
        self.y = torch.tensor(
            y, dtype=torch.float32
        )

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

Dataset görevi
Tek bir index için modelin ihtiyaç duyduğu örneği döndürmektir.

21. DataLoader

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_ds,
    batch_size=64,
    shuffle=True,
    num_workers=2
)
```

Parametre	Anlam
batch_size	Batch başına örnek
shuffle	Training sırasında örnek sırasını karıştır
num_workers	Paralel data loading
pin_memory	CUDA transferini hızlandırabilir
drop_last	Eksik son batch'i at

22. Train / validation modu

```
model.train()
# dropout aktif, batchnorm train mode

model.eval()
# dropout kapalı, batchnorm running stats
```

Kritik
Validation/inference öncesi model.eval() unutmak performansı ve tahminleri bozabilir.

23. Validation loop

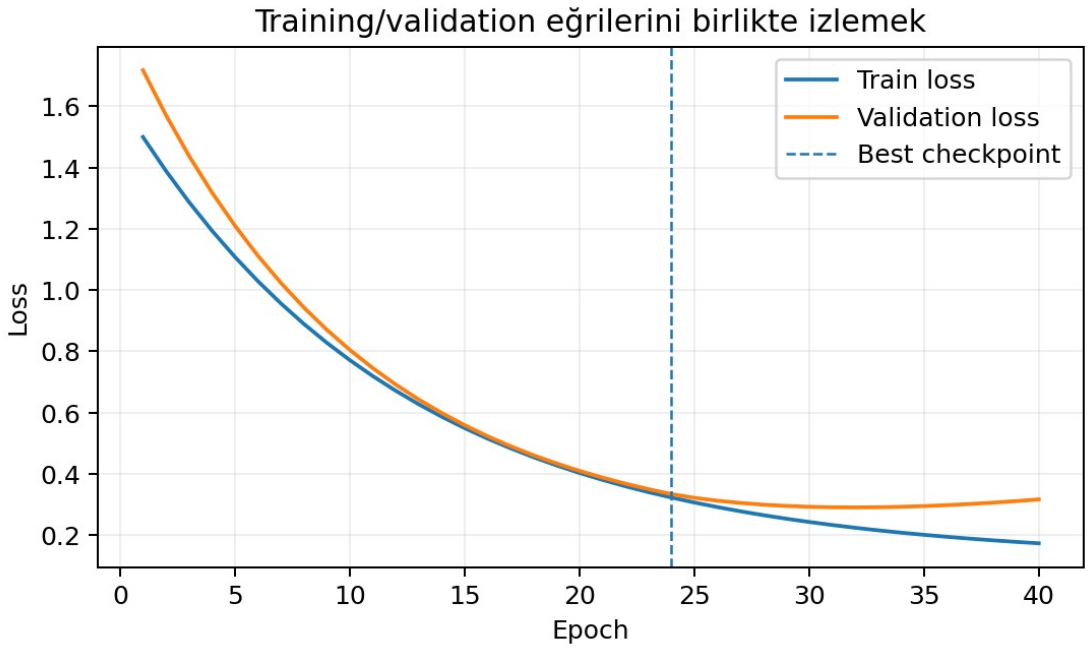
```
model.eval()
valid_loss = 0.0

with torch.inference_mode():
    for X_batch, y_batch in valid_loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        logits = model(X_batch)
        loss = criterion(logits, y_batch)

        valid_loss += loss.item()
```

24. Loss curve izleme



Pattern	Olası yorum
Train↓, Val↓	Öğrenme iyi
Train↓, Val↑	Overfitting
İkisi de yüksek	Underfitting / optimization sorunu
Loss NaN	LR, dtype, data veya numerical stability sorunu

25. Binary classification tam örnek

```
class BinaryMLP(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, 64),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.net(x).squeeze(1)

model = BinaryMLP(X_train.shape[1]).to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-3
)
```

Inference
Probability gerekiyorsa logits üzerine torch.sigmoid uygula.

26. Multiclass classification

```
model = nn.Sequential(
    nn.Linear(d, 128),
    nn.ReLU(),
    nn.Linear(128, n_classes)
)

criterion = nn.CrossEntropyLoss()
```

Target format
CrossEntropyLoss ile y genelde shape [batch] ve dtype torch.long olan class index'lerinden oluşur.

27. Regression

```
model = nn.Sequential(
    nn.Linear(d, 128),
    nn.ReLU(),
    nn.Linear(128, 1)
)

criterion = nn.MSELoss()
```

Scale
Regression target çok geniş ölçekteyse target transform/standardization optimization'ı kolaylaştırabilir.

28. CNN'e PyTorch ile giriş

```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU()
        )

    def forward(self, x):
        return self.features(x)
```

Tensor shape
Image tensor'ları PyTorch'ta çoğunlukla [batch, channel, height, width] düzenindedir.

29. Embedding katmanı

```
embedding = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=128
)

token_vecs = embedding(token_ids)
```

NLP bağlantısı
Embedding layer token id'lerini trainable dense vector'lara dönüştürür.

30. RNN/LSTM API fikri

```
lstm = nn.LSTM(
    input_size=128,
    hidden_size=256,
    batch_first=True
)

output, (h, c) = lstm(x)
```

Sonraki adım
31. modüldeki Transformer/LLM yapılarında sequence modeling attention ile ele alınacak.

31. Gradient clipping

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0
)
```

Ne zaman?
Exploding gradient riskinde, özellikle sequence modellerinde faydalı olabilir.

32. Learning-rate scheduler

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    patience=2,
    factor=0.5
)

scheduler.step(valid_loss)
```

Scheduler	Mantık
StepLR	Belirli epoch aralıklarında azalt
CosineAnnealing	Cosine schedule
OneCycleLR	Tek döngülü agresif schedule
ReduceLROnPlateau	Validation durunca azalt

33. Early stopping

Validation metric belirli patience boyunca iyileşmezse training durdurulur.

Checkpoint ile birlikte
En son epoch'u değil, en iyi validation checkpoint'ini saklamak tercih edilir.

34. Checkpoint kaydetme

```
torch.save(
    {
        "model_state": model.state_dict(),
        "optimizer_state": optimizer.state_dict(),
        "epoch": epoch,
        "valid_loss": valid_loss
    },
    "checkpoint.pt"
)
```

35. Checkpoint yükleme

```
ckpt = torch.load(
    "checkpoint.pt",
    map_location=device
)

model.load_state_dict(
    ckpt["model_state"]
)
```

state_dict
Model class/architecture code ile ağırlıkları ayrı tutmak PyTorch'un temel serialization yaklaşımıdır.

36. Reproducibility

```
import random
import numpy as np
import torch

seed = 42

random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

Tam determinism
GPU kernel'leri ve bazı parallel operation'lar nedeniyle aynı seed her zaman bit-level aynı sonuç garantilemez.

37. Mixed precision

FP16/BF16 ile daha az memory ve daha yüksek throughput elde edilebilir.

```
with torch.autocast(
    device_type="cuda",
    dtype=torch.float16
):
    logits = model(X_batch)
    loss = criterion(logits, y_batch)
```

Modern GPU
Mixed precision büyük modellerde training hızını ve batch capacity'yi ciddi artırabilir.

38. Gradient accumulation

GPU'ya büyük batch sığmadığında birkaç mini-batch gradient'i biriktirilerek effective batch büyütülebilir.

```
  loss = loss / accumulation_steps
  loss.backward()

  if step % accumulation_steps == 0:
     optimizer.step()
     optimizer.zero_grad()
```

39. Data augmentation

Training sırasında input'u anlamı koruyacak şekilde dönüştürerek generalization artırılabilir.

Alan	Augmentation
Image	crop, flip, color jitter
Audio	noise, time masking
Text	task-specific dikkatli augmentation
Tabular	Genellikle daha sınırlı ve domain bağımlı

40. Transfer learning

Pretrained bir modelin feature extractor veya initialization olarak kullanılmasıdır.

Strateji	Açıklama
Freeze backbone	Sadece head eğit
Partial unfreeze	Son katmanları fine-tune et
Full fine-tune	Tüm model düşük LR ile güncellenir

PyTorch ekosistemi
torchvision, torchaudio ve Hugging Face gibi kütüphaneler pretrained model kullanımını kolaylaştırır.

41. Custom training function

```
def train_one_epoch(
    model,
    loader,
    optimizer,
    criterion,
    device
):
    model.train()
    total = 0.0

    for X, y in loader:
        X, y = X.to(device), y.to(device)

        optimizer.zero_grad()
        out = model(X)
        loss = criterion(out, y)

        loss.backward()
        optimizer.step()

        total += loss.item()

    return total / len(loader)
```

42. Metric hesaplama

Loss optimization için, metric evaluation için kullanılır. Örneğin BCE loss ile train edip F1/ROC-AUC/PR-AUC raporlayabilirsin.

GPU tensor -> NumPy
Metric için tensor'ı önce detach().cpu() ile computation graph ve GPU'dan ayır.

```
probs = torch.sigmoid(logits)

y_score = (
    probs
    .detach()
    .cpu()
    .numpy()
)
```

43. Common PyTorch hataları

Hata	Kök neden
Expected all tensors same device	CPU/GPU mismatch
mat1 and mat2 shapes cannot be multiplied	Linear input_dim yanlış
Target dtype error	CrossEntropyLoss için y long değil
NaN loss	LR/numerical/data problemi
Validation değişken	model.eval() unutulmuş olabilir
Memory leak	Graph tutan tensor'ları listede saklamak

44. Debugging sırası

1. Bir batch'in shape/dtype/device değerlerini yazdır.
2. Tek batch üzerinde forward çalıştır.
3. Loss finite mi kontrol et.
4. Tek batch'e overfit etmeyi dene.
5. Gradient var mı kontrol et.
6. Learning rate'i kontrol et.
7. Train/eval mode doğru mu kontrol et.
8. Validation pipeline training'den bağımsız mı doğrula.

45. Tek batch'e overfit testi

Model küçük bir batch'i bile ezberleyemiyorsa architecture, loss veya training loop'ta bug olma ihtimali yüksektir.

En güçlü debug tekniklerinden
Önce 32 örneğe overfit et. Bu çalışmadan milyon satırlık training'e geçme.

46. Tabular churn vakası

Adım	PyTorch yaklaşımı
Input	Numeric + encoded categorical tensor
Model	MLP
Loss	BCEWithLogitsLoss
Imbalance	pos_weight / threshold
Metric	PR-AUC, recall, precision
Validation	Temporal veya stratified
Checkpoint	Best validation PR-AUC

Gerçekçi not
Tabular veride gradient boosting çoğu zaman güçlü baseline'dır. PyTorch kullanımı representation learning veya multimodal yapı varsa daha anlamlı olabilir.

47. Image classification vakası

Aşama	Karar
Dataset	ImageFolder/custom Dataset
Transform	resize + augmentation + normalize
Model	Pretrained CNN/ViT
Loss	CrossEntropyLoss
Optimizer	AdamW/SGD
Fine-tune	Backbone freeze -> unfreeze
Metric	Macro-F1 / accuracy

48. Text classification vakası

Token id -> embedding -> sequence encoder -> classification head akışı PyTorch ile kurulabilir.

Modern pratik
Transformer modelleri için çoğu veri bilimci PyTorch'u Hugging Face Transformers üzerinden kullanır.

49. Experiment tracking

- Seed
- Dataset version
- Architecture
- Hyperparameters
- Best checkpoint
- Train/validation curves
- Final metrics

MLOps bağlantısı
Daha sonra MLflow ve model registry ile bu metadata sistematik olarak takip edilecek.

50. Inference pipeline

```
model.eval()

with torch.inference_mode():
    logits = model(x.to(device))
    probs = torch.sigmoid(logits)

pred = (
    probs >= threshold
).long()
```

Preprocessing parity
Training'deki normalization/tokenization/encoding inference sırasında birebir aynı olmalıdır.

51. Model export seçenekleri

Format	Kullanım
state_dict	PyTorch-native en yaygın
TorchScript	Bazı production senaryoları
ONNX	Framework-independent serving
torch.compile	Runtime optimization yaklaşımı

52. Proje klasör yapısı

```
project/
  configs/
    train.yaml
  src/
    dataset.py
    model.py
    train.py
    evaluate.py
    inference.py
  checkpoints/
  notebooks/
  tests/
  requirements.txt
  README.md
```

Notebook'tan çıkış
Gerçek portföy projesinde training loop ve model class notebook hücrelerine gömülü kalmamalı.

53. Mülakat soruları

Soru	Kısa cevap
Tensor nedir?	Multi-dimensional numerical array + dtype/device/autograd metadata.
requires_grad ne yapar?	Autograd gradient takibini açar.
backward() ne yapar?	Computation graph üzerinden gradient hesaplar.
zero_grad neden?	Gradient accumulation'ı sıfırlar.
model.train vs eval?	Dropout/BatchNorm davranışını değiştirir.
Dataset ne yapar?	Index bazlı tek örnek döndürür.
DataLoader ne yapar?	Batching, shuffling ve parallel loading.
BCEWithLogitsLoss neden sigmoid'siz?	Sigmoid + BCE'yi numerically stable birleştirir.
CrossEntropy target dtype?	Genellikle torch.long.
state_dict nedir?	Model parameter/buffer dictionary'si.
inference_mode neden?	Gradient graph kapatır ve inference overhead'ini düşürür.
Device mismatch?	Model ve data farklı device'ta.
Mixed precision?	Lower precision ile memory/throughput avantajı.
Gradient clipping?	Gradient normunu sınırlar.
Single-batch overfit testi?	Training pipeline bug'larını hızlı bulmak için.

54. Uygulama alıştırmaları

1. NumPy array'i tensor'a çevirip shape/dtype/device işlemlerini dene.
2. Autograd ile y=x^3 fonksiyonunun gradient'ini hesapla.
3. Custom Dataset ve DataLoader yaz.
4. Binary classification için MLP kur ve BCEWithLogitsLoss ile eğit.
5. Validation loop ekle ve best checkpoint kaydet.
6. Aynı modeli CPU ve GPU'da çalıştırma kodunu yaz.
7. Gradient clipping ve LR scheduler ekle.
8. Tek batch'e overfit debugging testi yap.

9. Mixed precision training loop ekle.

10. state_dict ile modeli kaydedip yeni process mantığında yükle.

55. Mini portföy projesi

Proje: Production-Ready PyTorch Classification Pipeline

Dosya	İçerik
src/dataset.py	Custom Dataset/DataLoader
src/model.py	nn.Module architecture
src/train.py	Training/validation loop
src/metrics.py	PR-AUC/F1/threshold
src/inference.py	Checkpoint loading + predict
configs/train.yaml	Hyperparameter config
checkpoints/	Best model
tests/	Shape/device/data tests
README.md	Architecture + reproduce steps

CV cümlesi

Built a modular PyTorch training pipeline with custom Dataset/DataLoader components, GPU-aware training, checkpointing, mixed precision, validation monitoring and reproducible inference.

56. Final cheat sheet

Kavram	Hatırlanacak fikir
Tensor	Shape + dtype + device
Autograd	Automatic gradient
nn.Module	Model abstraction
Dataset	Tek örnek
DataLoader	Batch data flow
Forward	Prediction computation
Loss	Optimization objective
backward	Gradient calculation
optimizer.step	Parameter update
zero_grad	Gradient reset
train/eval	Layer behavior mode
inference_mode	No gradient inference
state_dict	Weights/buffers
CUDA	GPU acceleration
AMP	Mixed precision
Checkpoint	Best training state

Bu modülden sonra

31. Transformers & LLM bölümünde PyTorch tensor, embedding, attention, Dataset/DataLoader ve fine-tuning bilgisini doğrudan transformer mimarilerine taşıyacaksın.